

Erlind Sejdiu
106357

Exposé: Was kostet Observability?

Eine quantitative Analyse des Laufzeit- und Datenoverheads von OpenTelemetry

Seminararbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Betreuer: [Name der betreuenden Person]
Themengruppe: [Themengruppe des Seminars]

Inhaltsverzeichnis

1	Motivation	2
2	Problemstellung	2
3	Stand der Technik	3
4	Forschungsfrage	3
5	Methodik	4
5.1	Messaufbauten	4
5.2	Untersuchte Faktoren	4
5.3	Messgrößen	4
5.4	Versuchsdurchführung	5
5.5	Präsentation	5
6	Zeitplan	5
	Literaturverzeichnis	6

1 Motivation

Moderne Softwaresysteme bestehen zunehmend aus verteilten Diensten, deren Zusammenspiel sich ohne geeignete Werkzeuge kaum nachvollziehen lässt. Observability – die Fähigkeit, den inneren Zustand eines Systems anhand von Traces, Metriken und Logs zu verstehen – gilt daher als Voraussetzung für den zuverlässigen Betrieb solcher Systeme. OpenTelemetry hat sich hierfür als herstellerunabhängiger De-facto-Standard etabliert. Es ist eines der aktivsten Projekte der Cloud Native Computing Foundation und stellt APIs, SDKs und Werkzeuge für zahlreiche Programmiersprachen bereit [1], [2].

Observability ist jedoch nicht kostenlos. Jeder erzeugte Span benötigt Rechenzeit und Speicher, muss serialisiert, über das Netzwerk übertragen und schließlich gespeichert werden. Wie hoch diese Kosten sind, hängt stark davon ab, *wie* instrumentiert wird: Eine automatische Instrumentierung, die nur eingehende und ausgehende Anfragen erfasst, erzeugt wenige Spans pro Anfrage; wird hingegen jede Funktion mit einem eigenen Span versehen, steigt die Anzahl um Größenordnungen. Sampling-Verfahren versprechen, diese Kosten zu begrenzen, indem nur ein Teil der Traces aufgezeichnet wird.

Entwicklerinnen und Entwickler, die OpenTelemetry einführen, stehen damit vor der Frage, welcher Instrumentierungsgrad und welche Sampling-Strategie für ihr System vertretbar sind. Belastbare, vergleichbare Zahlen dazu sind jedoch schwer zu finden.

2 Problemstellung

Die Dokumentation von OpenTelemetry beschreibt ausführlich, *wie* Anwendungen instrumentiert werden können, macht aber nur wenige und stark kontextabhängige Angaben dazu, *welche Kosten* dabei entstehen [3]. Aussagen über den Overhead beruhen in der Praxis häufig auf Erfahrungswerten einzelner Unternehmen oder auf Messungen, deren Aufbau nicht nachvollziehbar ist. Insbesondere fehlt eine systematische Betrachtung, wie die Faktoren Instrumentierungsgrad, Sampling und Last *zusammenwirken*: Ein Overhead, der bei geringer Last vernachlässigbar ist, kann nahe der Kapazitätsgrenze eines Systems zu deutlich erhöhten Antwortzeiten führen.

Neben dem Laufzeit-Overhead wird die anfallende *Datenmenge* oft unterschätzt. Sie bestimmt maßgeblich die Kosten für Netzwerk, Speicher und Analyse-Backend und wächst mit der Granularität der Instrumentierung.

Abgrenzung. Die Arbeit untersucht den Overhead auf Seite der instrumentierten Anwendung sowie die anfallende Datenmenge. Nicht betrachtet werden der diagnostische Nutzen der Telemetrie, die Instrumentierung mobiler Clients sowie der Vergleich verschiedener Analyse-Backends hinsichtlich ihrer Funktionalität. Der Nutzen von OpenTelemetry für die Fehlerdiagnose in einem realen System ist Gegenstand einer separaten Projektarbeit des Autors an der HKA-App, für die diese Seminararbeit die Konfigurationsgrundlage liefert.

3 Stand der Technik

Verteiltes Tracing und Sampling. Googles Dapper zeigt, dass verteiltes Tracing im großen Maßstab mit geringem Overhead möglich ist, stützt sich dafür jedoch wesentlich auf aggressives Sampling und auf eine Instrumentierung weniger zentraler Bibliotheken [4]. Arbeiten wie Sifter [5] untersuchen, wie Sampling so gestaltet werden kann, dass trotz reduzierter Datenmenge aussagekräftige Traces erhalten bleiben. Der Overhead auf Seite der Anwendung steht dabei nicht im Mittelpunkt.

Overhead von Monitoring-Frameworks. Im Umfeld des Monitoring-Frameworks Kieker [6] wurde mit MooBench ein Mikrobenchmark entwickelt, der den Overhead von Instrumentierung auf Methodenebene systematisch misst und in die kontinuierliche Integration eingebunden werden kann [7]. MooBench zerlegt den Overhead in Anteile für Instrumentierung, Datenerfassung und Datenweitergabe und bietet damit eine etablierte Methodik, an der sich diese Arbeit orientiert. [Prüfen, ob es bereits MooBench-Vergleiche mit OpenTelemetry gibt (Arbeitsgruppe Hasselbring), und diese hier einordnen.]

Angaben von OpenTelemetry. Für den Java-Agent veröffentlicht OpenTelemetry Hinweise und Messergebnisse zum Performance-Overhead, weist jedoch selbst darauf hin, dass dieser stark von Anwendung, Last und Konfiguration abhängt [3].

Benchmark-Systeme und Messmethodik. Für realistische Messungen an Microservice-Systemen existieren Referenzanwendungen wie TeaStore [8], DeathStarBench [9] oder die OpenTelemetry Demo [10]. Für die statistisch belastbare Auswertung von Performance-Messungen, insbesondere auf der JVM mit ihrer Just-in-Time-Kompilierung, liegen methodische Empfehlungen vor [11], [12].

Forschungslücke. Es existieren sowohl Methoden zur Overhead-Messung als auch einzelne Messergebnisse zu OpenTelemetry. Weitgehend offen ist jedoch eine reproduzierbare Untersuchung, die das *Zusammenwirken* von Instrumentierungsgrad, Sampling und Last betrachtet und dabei neben dem Laufzeit-Overhead auch die anfallende Datenmenge quantifiziert.

4 Forschungsfrage

Aus der Problemstellung und dem Stand der Technik leitet sich folgende zentrale Forschungsfrage ab:

Wie wirken sich Instrumentierungsgrad, Sampling-Strategie und Last auf den Laufzeit-Overhead und die anfallende Telemetrie-Datenmenge eines mit OpenTelemetry instrumentierten Systems aus?

Daraus werden folgende Hypothesen abgeleitet, die in der Arbeit überprüft werden:

- H1:** Der Laufzeit-Overhead wächst annähernd linear mit der Anzahl erzeugter Spans pro Anfrage; eine Instrumentierung jeder Funktion verursacht daher einen um ein Vielfaches höheren Overhead als eine automatische Instrumentierung der Systemgrenzen.
- H2:** Der Overhead wirkt sich stärker auf hohe Perzentile der Antwortzeit (p95, p99) aus als auf den Median und nimmt mit steigender Last überproportional zu.
- H3:** Head-Sampling reduziert nicht nur die Datenmenge, sondern auch den Laufzeit-Overhead deutlich, da nicht aufgezeichnete Spans nur minimale Kosten verursachen.
- H4:** Ein wesentlicher Anteil des Overheads entsteht nicht beim Erzeugen der Spans, sondern bei deren Serialisierung und Export.

5 Methodik

Die Forschungsfrage wird experimentell beantwortet. Um sowohl präzise als auch realistische Aussagen zu erhalten, werden zwei sich ergänzende Messaufbauten verwendet.

5.1 Messaufbauten

Mikrobenchmark. In Anlehnung an MooBench [7] wird ein synthetischer Dienst verwendet, dessen Aufruftiefe und damit die Anzahl der Spans pro Anfrage exakt steuerbar ist. So lassen sich die Kosten eines einzelnen Spans isoliert bestimmen. Der Mikrobenchmark wird in zwei Laufzeitumgebungen umgesetzt, um deren Einfluss zu vergleichen: Java mit dem OpenTelemetry-Java-Agent (JIT-kompiliert, automatische Instrumentierung) und Go mit dem OpenTelemetry-SDK (nativ kompiliert, manuelle Instrumentierung). [\[Auswahl der Sprachen mit Betreuer abstimmen.\]](#)

Makrobenchmark. Als realistisches System wird eine etablierte Microservice-Referenzanwendung unter Last betrieben, voraussichtlich TeaStore [8], alternativ die OpenTelemetry Demo [10]. Die endgültige Auswahl erfolgt in der ersten Phase anhand der Kriterien Instrumentierbarkeit, Reproduzierbarkeit und Ressourcenbedarf.

5.2 Untersuchte Faktoren

Die Stufe „Export deaktiviert“ dient dazu, die Kosten für das Erzeugen von Spans von denen für Serialisierung und Export zu trennen (H4). Um die Anzahl der Messungen beherrschbar zu halten, werden nicht alle Kombinationen vollständig vermessen: Der Mikrobenchmark variiert Instrumentierungsgrad bzw. Aufruftiefe, Sampling und Export bei fester Last; der Makrobenchmark variiert Instrumentierungsgrad, Sampling und Last bei aktivem Export.

5.3 Messgrößen

- **Laufzeit:** Antwortzeit (Median, p95, p99), Durchsatz,
- **Ressourcen:** CPU-Auslastung und Speicherbedarf der Anwendung, bei Java zusätzlich Garbage-Collection-Aktivität,
- **Datenmenge:** Spans pro Anfrage, Bytes pro Span (OTLP), übertragenes Datenvolumen pro Sekunde, Speicherbedarf im Analyse-Backend hochgerechnet auf eine Million Anfragen,

Faktor	Stufen
Instrumentierungsgrad	G0: keine Instrumentierung (Baseline); G1: automatische Instrumentierung der Systemgrenzen (HTTP, Datenbank); G2: G1 zuzüglich manueller Spans für fachlich relevante Funktionen; G3: ein Span für jede Funktion
Sampling	100 %, 10 % und 1 % (Head-Sampling)
Export	aktiv (OTLP an Collector) vs. deaktiviert (Spans werden erzeugt, aber verworfen)
Last	25 %, 50 %, 75 % und 90 % des in G0 ermittelten maximalen Durchsatzes

Tabelle 1: Faktoren und Stufen der Messungen

- **Infrastruktur:** CPU- und Speicherbedarf des OpenTelemetry Collectors.

5.4 Versuchsdurchführung

Die Messungen werden auf dedizierter Hardware durchgeführt, um Störeinflüsse durch geteilte Cloud-Ressourcen zu vermeiden. Alle Komponenten werden containerisiert (Docker Compose), die Last wird reproduzierbar mit einem Lastgenerator (z.B. k6) erzeugt, und die Durchführung wird skriptgesteuert automatisiert. Gemäß den Empfehlungen von Georges et al. [12] wird jede Konfiguration nach einer Aufwärmphase mehrfach gemessen; die Ergebnisse werden mit Konfidenzintervallen angegeben. Messskripte, Konfigurationen und Rohdaten werden in einem Git-Repository veröffentlicht, sodass die Messungen nachvollziehbar und wiederholbar sind.

5.5 Präsentation

Die Ergebnisse werden in einem Abschlussvortrag vorgestellt. Kern des Vortrags sind wenige, aussagekräftige Visualisierungen, insbesondere der Overhead in Abhängigkeit von Spans pro Anfrage sowie von der Last, und daraus abgeleitete Empfehlungen für die Wahl von Instrumentierungsgrad und Sampling.

6 Zeitplan

Phase 1: Literaturrecherche und Versuchsplanung

Ziel: Aufarbeitung des Stands der Technik, Auswahl des Makrobenchmarks, Festlegung der Messgrößen.

Artefakt: Kapitel Grundlagen und Stand der Technik, Versuchsplan.

Phase 2: Aufbau der Messumgebung

Ziel: Umsetzung des Mikrobenchmarks, Inbetriebnahme des Makrobenchmarks, Automatisierung der Messungen.

Artefakt: Messumgebung im Git-Repository, Pilotmessung.

Phase 3: Durchführung der Messungen

Ziel: Vermessung aller geplanten Konfigurationen.

Artefakt: Rohdaten aller Messreihen.

Phase 4: Auswertung

Ziel: Statistische Auswertung, Überprüfung der Hypothesen, Ableitung von Empfehlungen.

Artefakt: Diagramme und Auswertungsskripte.

Phase 5: Ausarbeitung und Vortrag

Ziel: Schriftliche Ausarbeitung und Vorbereitung des Abschlussvortrags.

Artefakt: Seminararbeit und Präsentationsfolien.

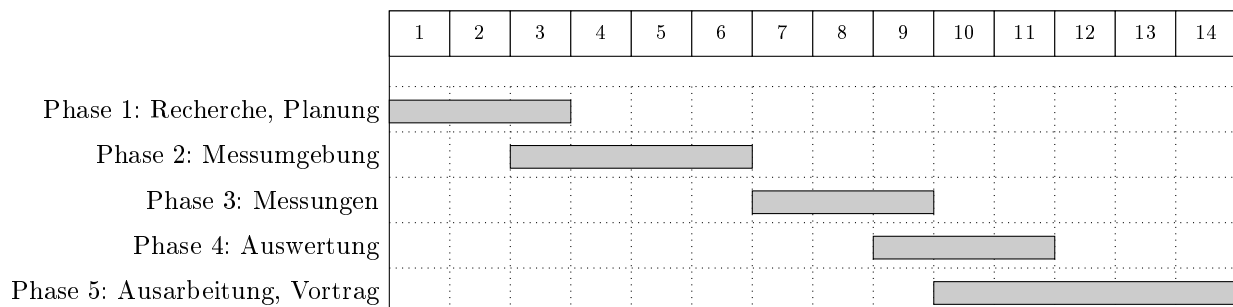


Abbildung 1: Zeitplan in Wochen [an Seminartermine anpassen]

Literaturverzeichnis

- [1] Cloud Native Computing Foundation. „OpenTelemetry – Project Page,“ besucht am 28. Sep. 2026. Adresse: <https://www.cncf.io/projects/opentelemetry/>
- [2] OpenTelemetry Authors. „OpenTelemetry Documentation,“ besucht am 28. Sep. 2026. Adresse: <https://opentelemetry.io/docs/>
- [3] OpenTelemetry Authors. „Java Agent Performance,“ besucht am 28. Sep. 2026. Adresse: <https://opentelemetry.io/docs/zero-code/java/agent/performance/>
- [4] B. H. Sigelman u. a., „Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,“ Google, Inc., Techn. Ber. dapper-2010-1, 2010.
- [5] P. Las-Casas, G. Papakerashvili, V. Anand und J. Mace, „Sifter: Scalable Sampling for Distributed Traces, without Feature Engineering,“ in *Proceedings of the ACM Symposium on Cloud Computing (SoCC '19)*, 2019, S. 312–324. DOI: 10.1145/3357223.3362736
- [6] A. van Hoorn, J. Waller und W. Hasselbring, „Kieker: A Framework for Application Performance Monitoring and Dynamic Software Analysis,“ in *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering (ICPE '12)*, 2012, S. 247–248. DOI: 10.1145/2188286.2188326

- [7] J. Waller, N. C. Ehmke und W. Hasselbring, „Including Performance Benchmarks into Continuous Integration to Enable DevOps,“ *ACM SIGSOFT Software Engineering Notes*, Jg. 40, Nr. 2, S. 1–4, 2015. DOI: 10.1145/2735399.2735416
- [8] J. von Kistowski, S. Eismann, N. Schmitt, A. Bauer, J. Grohmann und S. Kounev, „TeaStore: A Micro-Service Reference Application for Benchmarking, Modeling and Resource Management Research,“ in *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2018, S. 223–236. DOI: 10.1109/MASCOTS.2018.00030
- [9] Y. Gan, Y. Zhang, D. Cheng u. a., „An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems,“ in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, 2019, S. 3–18. DOI: 10.1145/3297858.3304013
- [10] OpenTelemetry Authors. „OpenTelemetry Demo,“ besucht am 28. Sep. 2026. Adresse: <https://opentelemetry.io/docs/demo/>
- [11] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. New York: John Wiley & Sons, 1991.
- [12] A. Georges, D. Buytaert und L. Eeckhout, „Statistically Rigorous Java Performance Evaluation,“ in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, 2007, S. 57–76. DOI: 10.1145/1297027.1297033